1. Introduction

These notes continue the study of sequential modeling architectures, building on the foundations of recurrent neural networks and the basic self-attention mechanism introduced in the previous lecture. Our main goals here are threefold. First, we study how to build rich, context-sensitive word representations that capture meaning as a function of surrounding text. Second, we examine two landmark language model architectures, BERT and GPT, which differ in their training objectives and architectural choices but share the Transformer as a backbone. Third, we broaden the concept of sequential modeling beyond text and consider how images can be treated as sequences, and finally we introduce a fundamentally different family of sequence models based on state space theory, culminating in the Mamba architecture.

A useful thread connecting all the material in these notes is the tension between two desiderata. On the one hand, we want a model that can capture arbitrarily long-range dependencies, so that a word or pixel far earlier in the sequence can still influence a later representation. On the other hand, we want a model that can be trained and evaluated efficiently. These two desiderata are often in conflict, and the different architectures studied here make different trade-offs between them.

2. From Static to Contextual Word Representations

Early neural network models for text relied on static word embeddings. A word such as "bank" was mapped to a single fixed vector, regardless of context. This is clearly problematic: the word "bank" in "river bank" and in "bank account" have entirely different meanings, yet a static embedding assigns them the same representation. The transition from static to contextual representations is one of the most important conceptual shifts in modern natural language processing.

In a static embedding scheme such as Word2Vec or GloVe, each word type is assigned a vector $e_w \in \mathbb{R}^d$. This vector is learned from co-occurrence statistics over large corpora, but once learned it is fixed. The fundamental limitation is that meaning is not a property of isolated words; it is a property of words in context. The sentence "She deposited the money at the bank" creates a different semantic environment around "bank" than "They fished along the bank." A good representation of "bank" in each sentence should reflect this difference.

Contextual word embeddings solve this by making the representation of each token a function of its entire surrounding context. Instead of a lookup table, we use a deep neural network that reads the full input sequence and produces a separate representation for each position. The representation of position i depends on all other positions, not just on the identity of the token at position i alone.

3. ELMo: Contextual Embeddings from Bidirectional Language Models

ELMo, which stands for Embeddings from Language Models, was one of the first successful contextual embedding methods. Its central idea is to train a deep bidirectional LSTM language model on a large corpus and then use the internal hidden states of that model as word representations.

A language model assigns probabilities to sequences of words. A forward language model factorizes the probability of a sequence $w_1, w_2, \ldots, w_N$ as

$$P(w_1, \ldots, w_N) = \prod_{k=1}^{N} P(w_k \mid w_1, \ldots, w_{k-1}).$$

At each step, the model sees the history and predicts the next word. A backward language model does the reverse, predicting each word from its future context:

$$P(w_1, \ldots, w_N) = \prod_{k=1}^{N} P(w_k \mid w_{k+1}, \ldots, w_N).$$

ELMo trains both directions simultaneously on the same corpus. A deep LSTM is used for each direction. In a network with L layers, each word at position k receives L hidden state vectors from the forward LSTM, L vectors from the backward LSTM, and the original token embedding. This gives a total of $2L + 1$ vector representations per token.

The question then becomes which of these vectors to use as the word representation for a downstream task. ELMo's answer is to take a learned weighted combination:

$$\text{ELMo}_k = \gamma \sum_{j=0}^{L} s_j \, h_{k,j},$$

where s_j are softmax-normalized scalar weights and γ is an overall scaling factor. Both s_j and γ are learned jointly with the downstream task. Different layers in a deep LSTM tend to capture different aspects of language: lower layers capture syntactic properties while higher layers capture more semantic information. By letting the task choose the mixture of layers, ELMo allows different tasks to exploit the level of abstraction most relevant to them.

ELMo demonstrated convincingly that context-sensitive representations could substantially improve performance on a range of NLP tasks. However, it was limited by the sequential nature of the LSTM. Training deep bidirectional LSTMs on large corpora is slow, and the representations are produced by a recurrent scan, which limits parallelism.

4. **BERT: Bidirectional Encoder Representations from Transformers**

BERT replaced the LSTM in ELMo with the Transformer encoder, removing the sequential computational bottleneck and replacing it with full attention over the entire input. BERT can be understood as the encoder half of the standard Transformer architecture applied to large-scale self-supervised pre-training.

Recall that in the original Transformer, the encoder processes the input sequence using stacked layers of multi-head self-attention and feedforward networks. Each layer updates the representation of every token by attending to all other tokens simultaneously. This is done using queries, keys, and values. For a single attention head with query matrix Q, key matrix K, and value matrix V, the attention output is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where d_k is the dimension of the keys. The division by $\sqrt{d_k}$ prevents the dot products from becoming too large in magnitude and causing the softmax to saturate. Multi-head attention applies this operation in parallel across h independently learned projection spaces and concatenates the results.

The key difference between BERT and earlier language models is bidirectionality. A standard left-to-right language model can only attend to the left context, because allowing attention to future tokens would trivially reveal the answer during training. BERT sidesteps this problem through a different training objective called masked language modeling.

5. **Training BERT**

BERT is trained with two objectives applied simultaneously. The primary objective is Masked Language Modeling (MLM). Before feeding a sentence to BERT, a fraction of the tokens (typically around fifteen percent) are randomly selected. Of those selected tokens, the majority are replaced with a special `[MASK]` token, a small fraction are replaced with a random word, and the rest are left unchanged. The model must then predict the original identity of each masked token. Concretely, the output representation at each masked position is passed through a linear layer followed by a softmax over the entire vocabulary. The training loss is the cross-entropy on the masked positions only.

The reason for partially leaving tokens unchanged and partially replacing them with random words is to prevent the model from learning to simply detect the `[MASK]` token during pre-training, since this token never appears at inference time. By sometimes presenting the original token or a random substitute, the model is forced to maintain useful representations for all tokens at all times.

The second objective is Next Sentence Prediction (NSP). For each training example, two sentences are drawn from the corpus. With probability one half they are consecutive sentences from the same document; otherwise the second sentence is drawn at random. BERT receives both sentences as a single sequence, separated by a special `[SEP]` token and preceded by a special `[CLS]` token. The representation at the `[CLS]` position is passed through a binary classifier that predicts whether the two sentences are consecutive or not. This objective was intended to help BERT learn inter-sentence relationships, which is useful for tasks like natural language inference and question answering.

Together, these two objectives allow BERT to be trained on raw text without any human annotation. This is the self-supervised pre-training paradigm. After pre-training on a large corpus, BERT can be fine-tuned on specific tasks with relatively small labeled datasets.

6. **Fine-Tuning BERT for Downstream Tasks**

After pre-training, BERT is adapted to downstream tasks by adding a small task-specific output layer and fine-tuning all parameters on labeled data. Different tasks require different output structures.

For classification tasks such as sentiment analysis, the representation at the `[CLS]` position is passed through a linear classifier. Because the `[CLS]` token attends to all positions in the sequence throughout all layers of BERT, its final representation summarizes the meaning of the entire input.

For token-level tasks such as part-of-speech tagging or named entity recognition, the output representation at each token position is passed through a shared linear layer that assigns a class label to each token independently. For tasks requiring pairwise sentence understanding, such as natural language inference, both sentences are concatenated into a single BERT input and the `[CLS]` representation is used for the final classification.

Extraction-based question answering provides a particularly elegant example. In tasks such as SQuAD, a passage and a question are concatenated into a single input. The model must identify the span within the passage that answers the question. This is handled by training two linear layers, one that scores each token as a potential start of the answer and one that scores each token as a potential end. The answer span is chosen as the pair (s, e) with the highest combined score $\alpha_s + \alpha_e$ subject to $s \leq e$. The ability to perform this span prediction with minimal task-specific architecture is a testament to the rich contextual representations produced by BERT.

7. **GPT: Decoder-Only Language Modeling**

While BERT uses the Transformer encoder and trains with a masked prediction objective, GPT takes the opposite architectural choice. GPT uses only the Transformer decoder and trains with a standard left-to-right language modeling objective: predict each token given all preceding tokens.

In the GPT architecture, each layer applies a causally masked self-attention operation. Causal masking means that the attention at position i is only allowed to attend to positions $j \leq i$. This ensures that the model cannot "see the future" when making predictions. After pre-training on large corpora, GPT generates text autoregressively: given a prompt, it samples one token at a time by computing a probability distribution over the vocabulary at each step and appending the sampled token to the context.

The conceptual difference between BERT and GPT reflects a deeper divide in their intended use cases. BERT is optimized for understanding tasks where the full input is available at inference time, and its bidirectional attention allows every token to be informed by every other token. GPT is optimized for generation tasks, where the output must be produced one token at a time without access to future tokens. Neither architecture is strictly superior; they excel in different settings. The historical trend has been toward scaling decoder-only models for general-purpose language understanding and generation, as exemplified by the GPT-3 and GPT-4 series.

8. **Vision Transformer**

The Vision Transformer, or ViT, demonstrates that the same architecture which works well for text sequences can also handle image data, provided we construct an appropriate sequential representation of images.

The key insight is that an image can be decomposed into a sequence of non-overlapping patches. Given an input image of size $H \times W$ with C channels and a chosen patch size $P \times P$, the image is divided into $N = HW/P^2$ patches. Each patch is a $P \times P \times C$ tensor, which is flattened into a vector of dimension P^2C and then linearly projected to a d-dimensional embedding space.

The resulting sequence of patch embeddings plays exactly the same role as a sequence of word embeddings in a text Transformer. A learnable class token `[CLS]` is prepended to the sequence, and a learnable positional encoding is added to every embedding to preserve spatial information. The entire sequence is then fed through a standard Transformer encoder. For classification, the output at the `[CLS]` position is passed through a linear head.

One important empirical finding is that ViT requires large amounts of training data to achieve competitive performance. When trained on ImageNet alone (roughly 1.2 million images), ViT underperforms well-tuned convolutional architectures such as EfficientNet. However, when pre-trained on much larger datasets such as JFT-300M, ViT matches or exceeds the performance

of the best convolutional models. This suggests that the inductive biases built into convolutional networks (local receptive fields, translation equivariance) are helpful with limited data but become less important at very large scale.

Several improved ViT variants address the data efficiency and computational cost. The Swin Transformer introduces hierarchical feature maps and computes attention within local windows that shift between layers. This reduces the quadratic cost of full attention and enables the model to process high-resolution inputs, making it suitable for dense prediction tasks such as object detection and semantic segmentation.

9. **The Quadratic Bottleneck and State Space Models**

The standard self-attention mechanism has a computational cost that scales as $O(N^2)$ in the sequence length N. For a sequence of N tokens, the attention matrix has N^2 entries, each of which requires an inner product computation. When N is small, this is not a problem. But as sequences grow longer, the quadratic cost becomes prohibitive in both memory and computation.

This is a fundamental limitation for applications requiring very long context windows. A GPT-3 model supports a context window of around 4096 tokens. GPT-4 can handle up to 32768 tokens. But even these lengths are modest compared to, say, a full book, a long video, or a genomic sequence. To go truly long, we need architectures whose cost scales sub-quadratically with sequence length.

One principled alternative is state space models, or SSMs. The foundational idea comes from control theory and signal processing. A continuous-time linear state space model is defined by the equations

$$\begin{aligned} h'(t) &= Ah(t) + Bx(t), \\ y(t) &= Ch(t) + Dx(t), \end{aligned}$$

where $x(t) \in \mathbb{R}$ is the input signal at time t, $h(t) \in \mathbb{R}^N$ is a hidden state vector, and $y(t) \in \mathbb{R}$ is the output. The matrix $A \in \mathbb{R}^{N \times N}$ governs how the state evolves over time, $B \in \mathbb{R}^{N \times 1}$ maps input to state, $C \in \mathbb{R}^{1 \times N}$ maps state to output, and $D \in \mathbb{R}$ is a direct skip connection. The state $h(t)$ plays the role of memory: it summarizes the history of the input up to time t in a compressed form.

To use such a model on discrete sequences, we must discretize the continuous-time equations. One natural approach is Euler discretization with step size Δ:

$$h_t \approx (I + \Delta A)h_{t-1} + \Delta B x_t, \quad y_t = Ch_t + Dx_t.$$

Let $\bar{A} = I + \Delta A$ and $\bar{B} = \Delta B$. Then the model becomes a linear recurrence:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t + Dx_t.$$

This recurrence can be unrolled as a linear RNN. It has the same infinite context property as a standard RNN: each hidden state h_t depends, in principle, on all previous inputs. But crucially, since the recurrence is linear, it can be equivalently expressed as a convolution. Define the kernel

$$\bar{K} = (C\bar{B},\ C\bar{A}\bar{B},\ C\bar{A}^2\bar{B},\ \ldots),$$

then the output sequence satisfies $y = x * \bar{K}$, where $*$ denotes convolution. During training, convolutions can be computed in $O(N \log N)$ time using the Fast Fourier Transform, enabling parallelism. During inference, one can switch to the recurrent form, evaluating each new token in $O(N)$ time with respect to the state dimension, independent of the sequence length. This dual-mode computation is a key practical advantage of SSMs over Transformers.

10. **Structured State Space Models and the Role of the A Matrix**

A naive implementation of the SSM described above requires computing powers of the matrix A, which is expensive when A is large and dense. The Structured State Space Model, or S4, proposes to initialize A using a special structure called HiPPO (High-order Polynomial Projection Operators). The HiPPO matrix is designed so that the hidden state $h(t)$ maintains an optimal polynomial approximation of the input history over an exponentially-decaying window. In other words, the matrix A is not arbitrary; it is chosen to encode the inductive bias that the model should remember the past efficiently.

With the HiPPO initialization, S4 demonstrates that SSMs can match Transformers on tasks requiring very long-range dependencies, such as the Long Range Arena benchmark, while maintaining linear computational complexity in sequence length. The structured parameterization also makes it possible to compute the convolution kernel $\bar{K}$ efficiently using special mathematical techniques such as the Cauchy kernel representation.

11. **Mamba: Selective State Space Models**

A fundamental limitation of S4 and related SSMs is that the matrices A, B, C, and Δ are fixed for all inputs once training is complete. The model applies the same linear transformation to every input token at every position. This means the model cannot selectively focus on some inputs and ignore others in the way that attention can. If the model encounters an irrelevant token, it has no mechanism to gate it out.

Mamba addresses this by making the parameters B, C, and Δ functions of the input. Concretely, for each input x_t, the model computes

$$B_t = \text{Linear}_B(x_t), \quad C_t = \text{Linear}_C(x_t), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t)),$$

where Linear_* denotes a learned linear projection and softplus ensures that $\Delta_t > 0$. This input-dependence is called the selection mechanism. It allows the model to decide, at each step, how strongly to incorporate the current input into the state and how strongly to read from the state for the current output.

The discretized state transition now uses time-varying parameters:

$$\bar{A}_t = \exp(\Delta_t A), \quad \bar{B}_t = (\Delta_t A)^{-1}(\exp(\Delta_t A) - I)\Delta_t B_t,$$

where A itself remains fixed (and structured via HiPPO). The time-varying nature of $\bar{A}_t$ and $\bar{B}_t$ breaks the convolution equivalence from S4, since a convolution requires a fixed kernel. The recurrence must therefore be computed sequentially, which would be slow in practice.

To recover parallelism, Mamba uses the parallel scan algorithm. This algorithm observes that a sequence of linear recurrences $h_t = a_t h_{t-1} + b_t$ can be evaluated in $O(\log N)$ parallel steps using a tree of pairwise reductions. At each level of the tree, adjacent pairs are combined, and the total work is $O(N)$ with $O(\log N)$ depth. This allows Mamba to train efficiently on hardware despite the input-dependent recurrence.

In addition, Mamba employs a hardware-aware implementation that avoids materializing the full hidden state sequence in high-bandwidth memory. By performing the scan in on-chip SRAM (fast memory) rather than HBM (slow memory), Mamba achieves significantly better throughput than a naive implementation.

The overall Mamba block interleaves the selective SSM with linear projections and nonlinearities following a gated convolutional design. Empirically, Mamba achieves performance comparable to

Transformers on language tasks while maintaining linear complexity in sequence length, making it attractive for long-context applications.

12. **Vision Mamba**

Following the success of ViT, which applies the Transformer to image patches, several works have explored replacing the Transformer with Mamba in vision tasks. The motivation is the same as in the language domain: Mamba's linear complexity in sequence length is attractive when processing high-resolution images, which may contain thousands of patches.

The primary challenge in applying Mamba to images is that images are two-dimensional while Mamba processes one-dimensional sequences. Vision Mamba and related models address this by scanning the image patches in multiple directions. For example, patches may be scanned from top-left to bottom-right, from bottom-right to top-left, and in column-major order. By running the SSM in all directions and aggregating the results, the model can capture both horizontal and vertical dependencies.

LocalMamba refines this idea by performing local window scans within non-overlapping image regions, similar in spirit to the Swin Transformer. By restricting the scan to local windows, the model avoids unnecessary long-range computation for tasks where local context is sufficient.

An important empirical question is whether Mamba's advantages are genuine for vision tasks. The paper MambaOut provides a careful study showing that for tasks such as image classification and object detection, the gating and state-space components of Mamba can be matched by simpler gated convolutional architectures without a recurrent state. This suggests that the benefit of selective state spaces may be most pronounced for tasks requiring truly long-range sequential reasoning, such as long video understanding or genomics, rather than image classification where local features dominate.

13. **Concluding Remarks**

The progression from static word embeddings to ELMo to BERT and GPT reflects a gradual shift toward architectures that can represent meaning as a context-sensitive function of an entire sequence. ELMo demonstrated the value of deep bidirectional representations. BERT showed that pre-training on a masked prediction task with the Transformer encoder produces highly general representations. GPT demonstrated that a decoder-only Transformer trained to predict the next token can generate coherent and knowledge-rich text.

The extension of sequential modeling to vision, through ViT, showed that the Transformer is not a specialized text architecture but a general-purpose sequence processor. Any domain in which data can be serialized into tokens can potentially benefit from this approach.

State space models, culminating in Mamba, offer a compelling alternative that avoids the quadratic attention bottleneck. The key conceptual contributions are threefold: the HiPPO initialization for principled memory structure, the selection mechanism for input-dependent filtering, and the parallel scan algorithm for efficient training. The relationship between SSMs and RNNs on one side, and between SSMs and convolutions on the other, reveals a deep unity among different families of sequence models that is worth appreciating.

When studying these architectures, it is useful to keep in mind the two fundamental questions they each answer: how is context incorporated (locally or globally, bidirectionally or causally),

and at what computational cost (quadratic, linear, or constant per token during inference). These two dimensions largely determine the practical trade-offs of each approach.